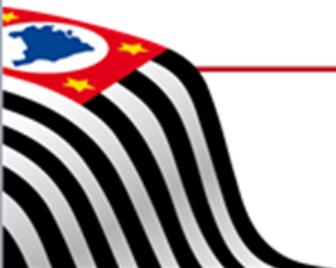
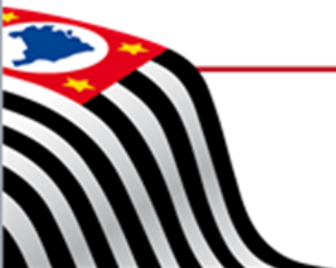


Classes, Objetos e Métodos



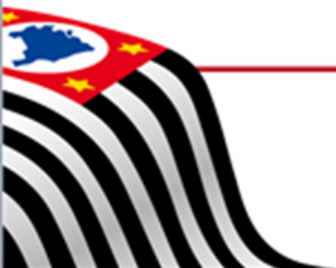
Objetivos

- Declarar uma classe e utilizá-la
- Declarar métodos e utilizá-los.
- Métodos com parâmetros
- Classe Scanner
- Método printf
- Método set
- Método get
- Referência this



Classe

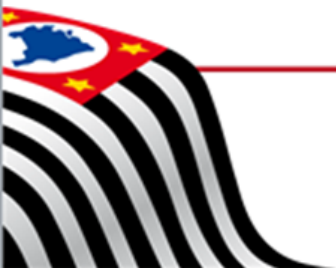
É um projeto que define objetos descrevendo seus atributos e métodos.



Classe

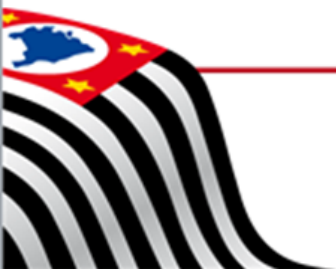
Exemplos

- Planta de uma casa
- Projeto de um automóvel



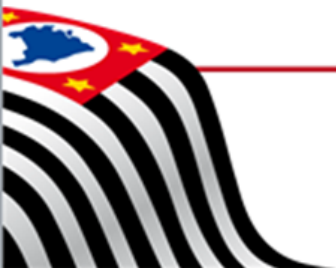
O que é um objeto

Objeto é uma extensão do conceito de objeto do mundo real.



Exemplo

- Pessoa
- Casa
- Automóvel
- Computador
- Relatório

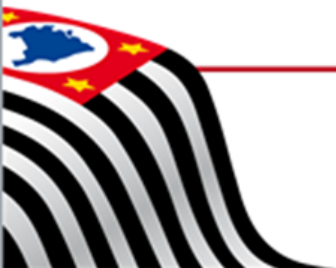


Declaração de classe

Palavra-chave **class**

Introduz uma declaração de classe.

É seguida pelo nome da classe.



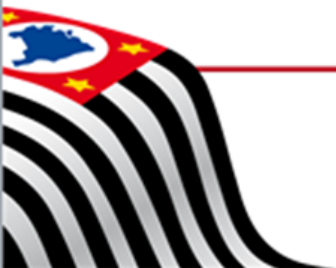
Declaração de classe

- Nomes de classes iniciam com uma letra maiúscula.

Exemplo: **class Exercicio**

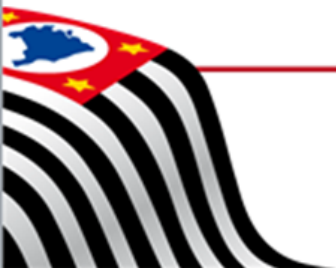
- Cada palavra inserida inicia em maiúscula.

Exemplo: **class ExercicioTestePrograma**



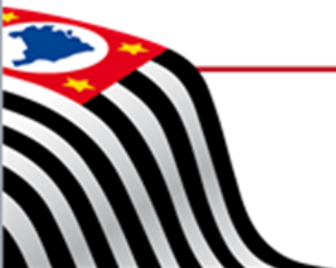
Declaração de classe

- { chave esquerda inicia o corpo de cada declaração de uma classe.
- }
- } chave direita finaliza a declaração de uma classe.



Métodos

Definem as funcionalidades da classe, ou seja, o que será possível fazer com objetos dessa classe.



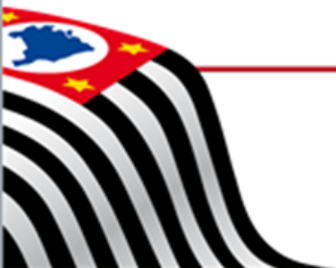
Métodos

Exemplo:

Veículo – Acelerador – aciona uma tarefa.

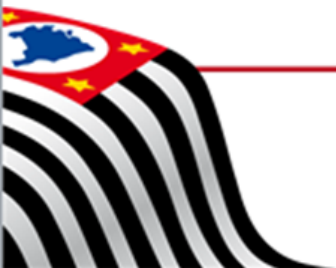
Enviar mensagem para um objeto

Cada mensagem é implementada como uma **chamada de método** que instrui um método do objeto a realizar sua tarefa.



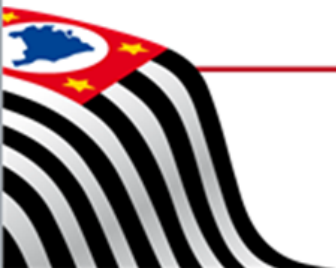
Declaração de um método

```
[modificador] tipo_retorno identificador ([parâmetros])  
{  
    corpo do método  
}
```

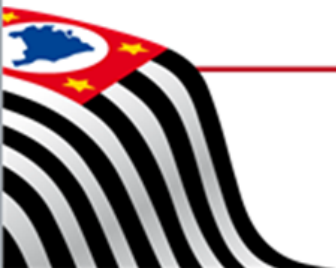


Declaração de um método

- **Modificador:** public.
- **Tipo de retorno:** tipo de dado que o método retorna após a realização de uma tarefa.



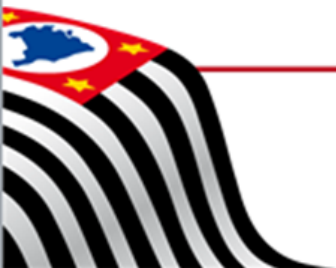
Para todo método cujo tipo de retorno seja diferente de void, deve-se especificar o tipo de retorno e no corpo do método, solicitar o retorno de um valor ou variável com o tipo especificado, usando a instrução **return**.



Declaração de um método

- **Identificador:** Nomes de métodos iniciam com a primeira letra minúscula e as palavras subsequentes do nome iniciam com letra maiúscula.

Exemplo: **exibeMensagem**

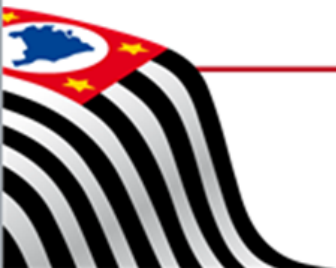


Declaração de um método

O Corpo do método:

É delimitado pelas chaves { e }

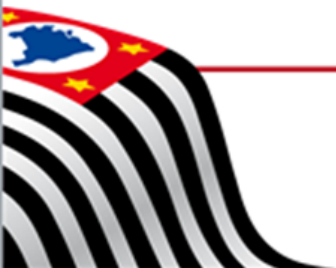
Contém uma ou mais instruções que realizam a tarefa do método.



Declaração de um método

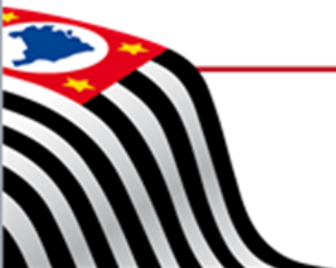
Exemplo de declaração de método sem retorno:

```
public void exibeMensagem ( )
```



Exemplo

- Criar a classe Curso com o método exhibeMensagem
- Criar a aplicação CursoTeste que instancie um objeto e chame o método exhibeMensagem



//Declaração de classe com um método

public class Curso

{

//Método que exibe uma mensagem

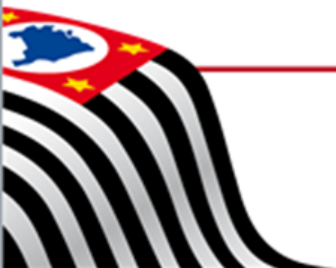
public void exibeMensagem ()

{

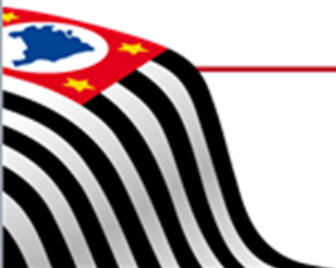
System.out.println ("Bem vindo ao primeiro módulo");

}// fim do método

} // fim da classe



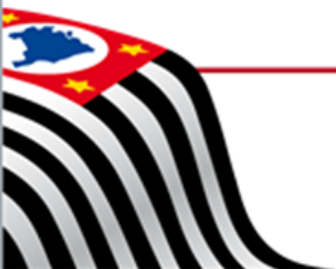
```
// Criando um objeto Curso e chamando seu método  
    exhibeMensagem  
public class TesteCurso  
{  
    // método main inicia a execução do programa  
    public static void main(String[] args)  
    {  
        // cria um objeto Curso e o atribui a meuCurso  
        Curso meuCurso = new Curso ( );  
        // chama método exhibeMensagem de meuCurso  
        meuCurso.exibemensagem( );  
    } // fim de main  
} //fim da classe
```



Programando

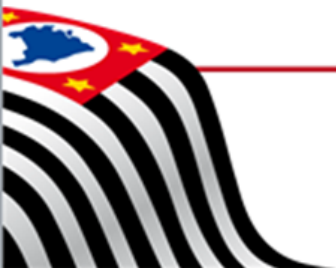


- Criar a classe Correntista contendo os atributos nome e saldo. Criar o método atualizaCadastro que deve exibir uma mensagem de atualização.
- Elaborar uma aplicação que crie uma instância da classe Correntista e chame o método atualizaCadastro.



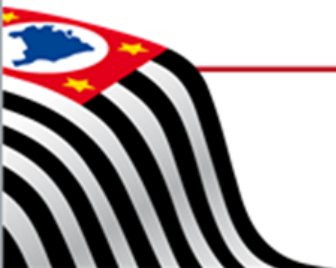
Método main

```
public static void main (String [ ] args)
```



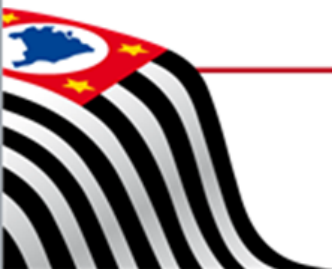
Método main

O método main é chamado automaticamente
pela Java Virtual
Machine quando executamos um aplicativo



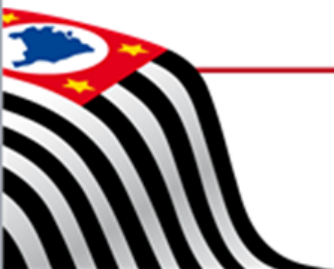
Método com parâmetro

Parâmetro: informações adicionais para execução do método.

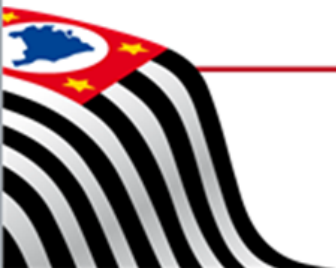


Método com parâmetro

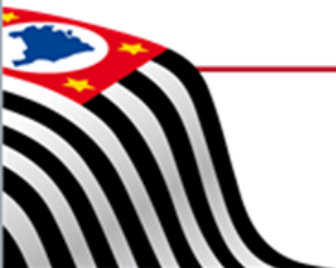
Parâmetros: são definidos em uma lista de parâmetros separados por vírgula, que está localizada no parênteses.



- Os parênteses **vazios** que se seguem ao nome de método indicam que um método não requer **nenhum parâmetro**.

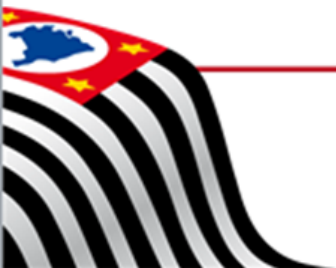


Uma chamada de método fornece valores – chamados **argumentos** – para cada um dos parâmetros do método.

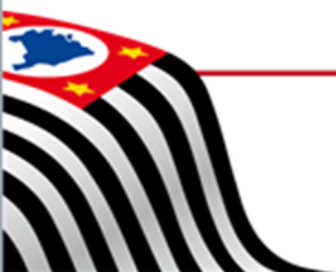


Exemplo:

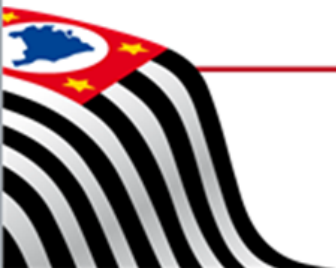
Classe Curso com um método `exibeMensagem` que mostra o nome do curso como parte da mensagem de boas-vindas



```
public class Curso {  
    public void exibeMensagem(String nomeCurso)  
    {  
        System.out.println ("Bem-vindo ao primeiro módulo: " +  
        nomeCurso );  
    }  
}
```



```
public class TesteCurso {  
  
    public static void main(String[] args) {  
        String nCurso = "Java";  
        Curso meuCurso = new Curso();  
        meuCurso.exibeMensagem (nCurso);  
    }  
}
```



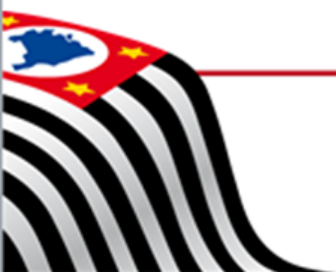
Programando

- Criar a classe Cliente contendo os atributos nome, número da conta e saldo.

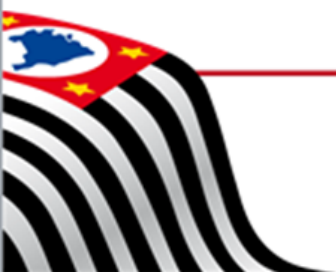
Inicializar saldo com um valor.

Criar o método `exibeSaldo` que deve exibir o saldo atualizado do cliente (acrescentar o depósito).

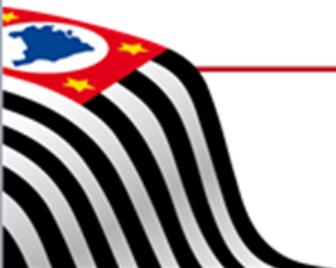
- Elaborar uma aplicação que crie uma instância da classe Cliente, receba o valor do depósito e chame o método `exibeSaldo`.
- Utilizar a classe Scanner e o método `printf`.



```
public class Cliente {  
    public String nome;  
    public int numconta;  
    public double saldo = 100.00;  
    public void exhibeSaldo(double deposito){  
        System.out.println("Saldo atualizado: " + (saldo + deposito));  
    }  
}
```



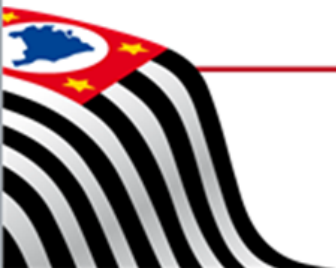

```
public class TesteCliente {  
    public static void main(String [ ] args) {  
        double sal;  
        sal=1000.00;  
        Cliente A = new Cliente ( );  
        A.exibeSaldo(sal);  
    }  
}
```



Classe Scanner

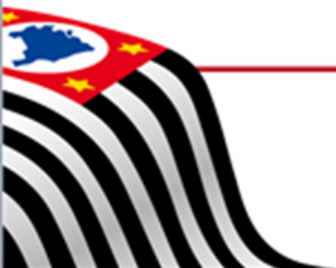
Importar a classe:

```
import java.util.Scanner;
```



Criar instância da classe

Scanner entrada = new Scanner(System.in);



Método **next<tipo>()**

Atribui ao objeto o valor digitado pelo usuário.

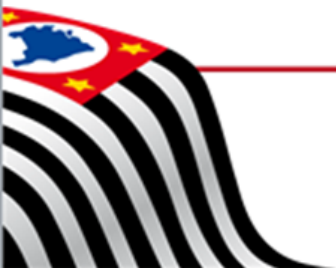
int **nextInt()**;

float **nextFloat()**;

double **nextDouble()**;

char **nextChar()**;

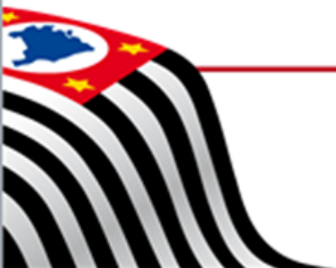
String **nextLine()**;



```
import java.util.Scanner;

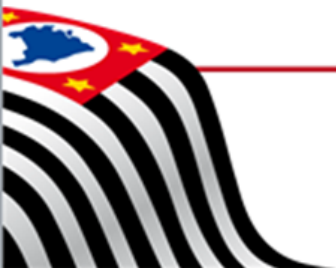
public class Exemplo
{
    public static void main (String args [ ] )
    {
        String nome;

        Scanner entrada = new Scanner (System.in);
        System.out.println ("Digite o seu nome: ");
        nome = entrada.nextLine();
        System.out.println ("O nome digitado foi: " + nome);
    }
}
```

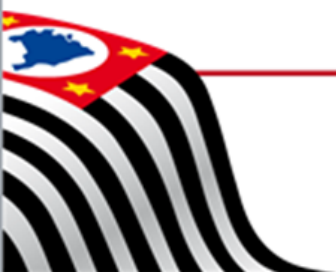


Exemplo

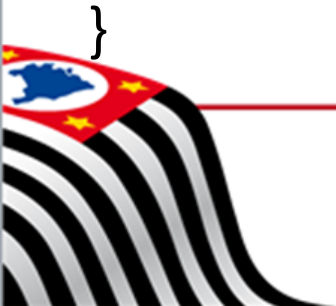
Classe Curso com um método
exibeMensagem que **insere** e mostra o
nome do curso como parte da mensagem
de boas-vindas.



```
public class Curso {  
    public void exibeMensagem(String nomeCurso)  
    {  
        System.out.println ("Bem-vindo ao primeiro módulo: " +  
        nomeCurso );  
    }  
}
```

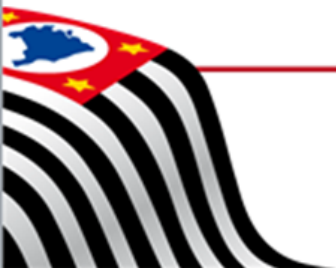


```
import java.util.Scanner;
import org.me.cursoCurso;
public class TesteCurso {
    public static void main(String [ ] args) {
        String nCurso;
        Scanner entrada = new Scanner (System.in);
        Curso meuCurso = new Curso();
        System.out.println("Digite o nome do curso");
        nCurso = entrada.nextLine( );
        meuCurso.exibeMensagem (nCurso);
    }
}
```



Método printf

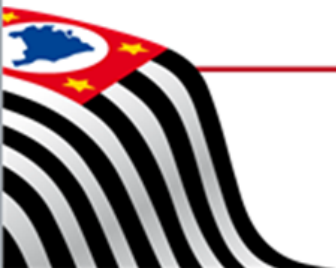
- Método que exibe os dados formatados.



Método printf

O método printf tem a forma:

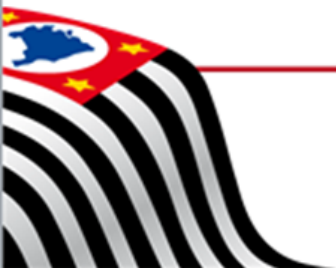
printf (string-de-formato , lista de argumentos);



Método printf

A string de formato pode consistir em texto fixo e especificadores de formato.

O especificador de formato inicia com um sinal de % e é seguido por um caractere de conversão que representa o tipo de dado do valor a ser impresso.

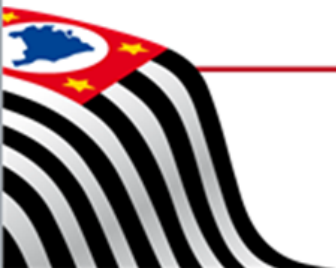


Especificadores de Formato

%d: Exibe um inteiro

%f: Exibe um valor de ponto flutuante no formato decimal.

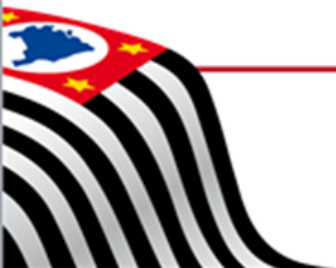
%s: Formata o argumento como uma String



Especificadores de Formato

`%g`: Exibe um valor de ponto flutuante em notação científica.

`%`: Insere na string o caractere de porcentagem.



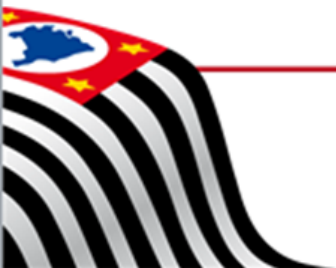
Método printf

Exemplo:

```
System.out.printf("%s\n%s\n", "Bem-vindo ao", "Curso de Java");
```

Saída:

Bem-vindo ao
Curso de Java

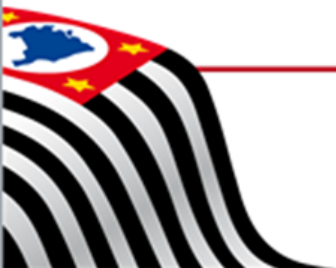


Método printf

Exemplo:

```
System.out.printf("Imprimindo número %d\n", 10);
```

```
System.out.printf("Resultado da soma %.2f\n", soma);
```

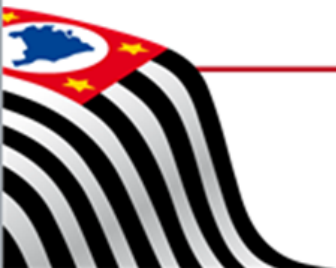


Definindo métodos

Encapsulamento

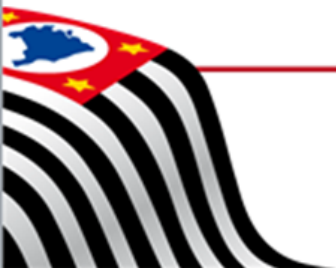
Atributos da classe definidos como private.

Criar um conjunto de métodos para manipulá-los.



Método set

Altera o valor do atributo de instância

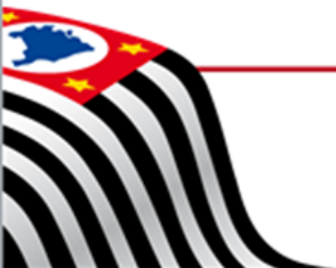


Método do tipo set

Altera o valor do atributo de instância.

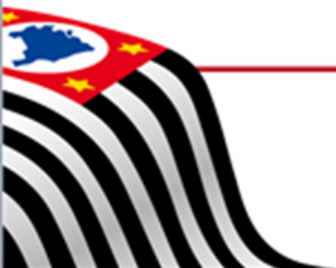
Se não criar o método set para um atributo, este não deve ser modificado.

- Método do tipo set não tem retorno – **void**
- Tem que receber um argumento – novo valor do campo.



Método set

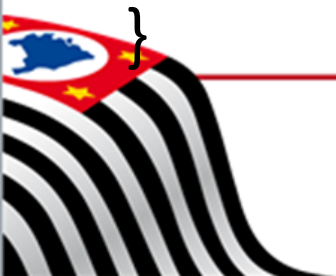
```
public void setNomeMétodo (parâmetros)
{
    <atributo> = <argumento>;
}
```



Método set - Exemplo

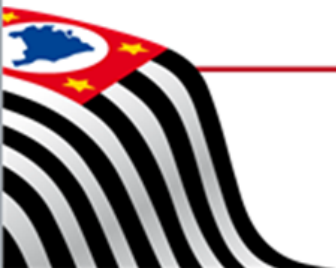
```
public class Curso {  
    private String nomeCurso;
```

```
        public void setNomeCurso (String nome)  
        {  
            nomeCurso = nome;  
        }  
    }
```



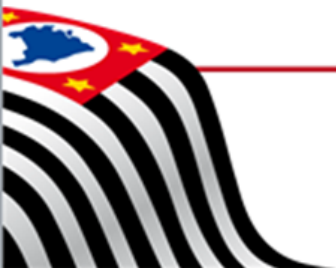
Método get

Retorna o valor do atributo de instância.



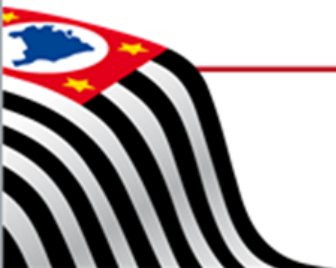
Método tipo get

- Verifica um valor e tem um retorno
- Não tem nenhum argumento.



Método get

```
public <tipo de dado> getNomeMétodo ( )  
{  
    return atributo;  
}
```

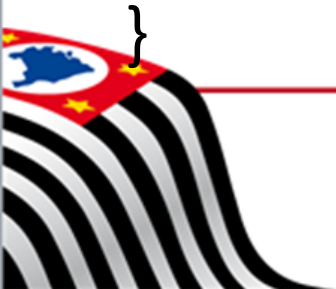


Método get - Exemplo

```
public class Curso {  
    private String nomeCurso;
```

```
    public String getNomeCurso ( )  
    {  
        return nomeCurso;  
    }
```

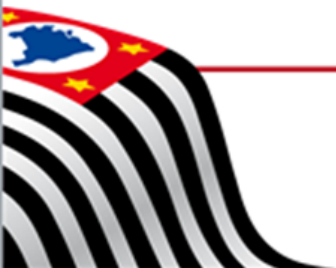
```
}
```



Exemplo

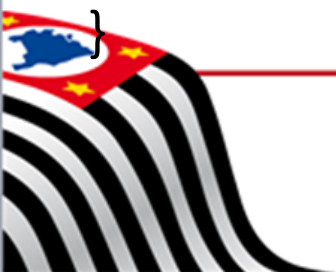
Cria a classe Curso com o atributo privado nomeCurso.

Cria os métodos get e set.



```
public class Curso {  
    private String nomeCurso;  
  
    public void setNomeCurso(String  
    nome)  
    {  
        nomeCurso = nome;  
    }  
    public String getNomeCurso ( )  
    {  
        return nomeCurso;  
    }  
}
```

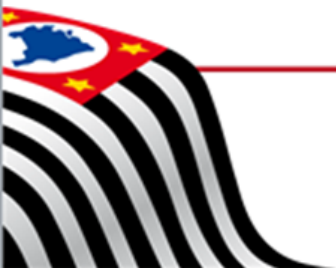
```
import java.util.Scanner;
import org.me.cursoCurso;
public class TesteCurso {
    public static void main(String[] args) {
        String nomec;
        Scanner entrada = new Scanner (System.in);
        Curso meuCurso = new Curso( );
        System.out.println("Digite o nome do curso: " );
        nomec = entrada.nextLine();
        meuCurso.setNomeCurso(nomec);
        System.out.println( );
        System.out.println("Curso de: " + meuCurso.getNomeCurso ());
    }
}
```



Referência this

Trata-se da referência do próprio objeto.

Faz uma diferenciação
entre
atributos e parâmetros de métodos.

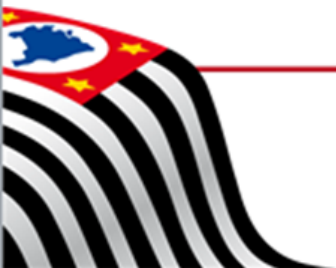


Exemplo

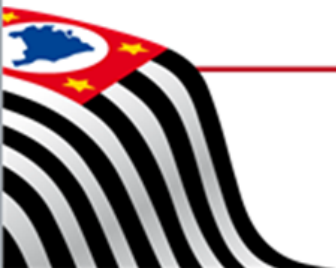
```
public void setNome(String nome) {  
    this.nome = nome;  
}
```

// **this.nome** refere-se ao atributo de instância nome

// **nome** refere-se ao parâmetro
}



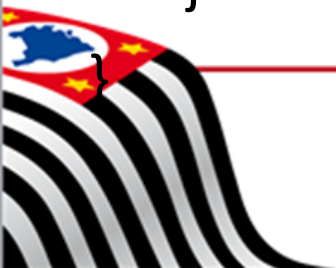
Exemplo do uso da referência this



```
public class Pessoa {  
    private String nome;
```

```
    public void setNome(String nome)  
{  
        this.nome = nome; // uso da referência this  
}
```

```
    public String getNome() {  
        return nome;  
    }
```



```
public class Pessoa {  
    private String nome;  
    public void setNome(String n)  
    {  
        nome = n;    // sem uso da referência this  
    }  
  
    public String getNome ( )  
    {  
        return nome;  
    }  
}
```

